Arquitectura de Computadoras 2026-1
Tarea 3

April 29, 2026

Expediente	Nombre
223210350	Amaya Soria Angel Alberto
219208106	Bórquez Guerrero Angel Fernando
223215039	Miranda Sánchez Javier Leonardo
223203899	Tostado Cortes Dante Alejandro

---

1. Suponer una CPU superescalar con scheduling dinámico con las siguientes características:

   - Emisión: 1 instrucción por ciclo.
   - Compromiso: 1 instrucción por ciclo.
   - ROB con suficientes entradas.
   - Estaciones de reserva suficientes.
   - Latencias: lw (2 ciclos), add (2 ciclos), mul (4 ciclos).

   Usando el algoritmo de Tomasulo, simular la ejecución del siguiente programa y llenar la tabla:

Instrucción	Emisión	Ejecución	Escritura	Compromiso
`lw f2, 0(r1)`	1	2 - 3	4	5
`lw f4, 0(r2)`	2	3 - 4	5	6
`mul f6, f2, f4`	3	5 - 8	9	10
`add f2, f6, f4`	4	9 - 10	11	12
`add f8, f2, f4`	5	11 - 12	13	14
`mul f10, f8, f2`	6	13 - 16	17	18
`add f6, f2, f8`	7	13 - 14	15	19
`add f12, f6, f10`	8	17-18	19	20

2. La figura 5.44 (página 472 del libro Computer Organization and Design)

   indica que la política de escritura (write policy) del caché primario (L1) de ARM Cortex-A8 es "write-back, write-allocate". Investigar esta política y describirla brevemente.

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement	–	Approximated LRU
L3 block size	–	64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles

Figure 5.44: Caches in the ARM Cortex-A8 and Intel Core i7 920.

write-back

Cuando el procesador escribe un dato, la modificación se realiza únicamente en el caché. El bloque modificado se marca con un dirty bit. La escritura de memoria principal ocurre solo cuando el bloque es desalojado del caché. Esto reduce el tráfico al bus de memoria comparado con **write-through**, ya que múltiples escrituras al mismo bloque generan una sola escritura a memoria.

write-allocate

Cuando ocurre un write miss, se carga el bloque completo desde memoria principal al caché antes de realizar la escritura. Así, escrituras futuras al mismo bloque encontrarán el dato en caché. Esta política es coherente con **write-back**, ya que al traer el bloque al caché se aprovecha la localidad espacial y temporal.

Esta combinación es especialmente eficiente en cargas de trabajo con alta localidad temporal de escrituras, ya que minimiza la latencia de escritura y el tráfico hacia niveles inferiores de la jerarquía de memoria.

3. Explicar con palabras propias y proporcionar un ejemplo de código (C, Java o Python) para cada caso.

a) Localidad temporal.
La **localidad temporal** se refiere al principio de que si un dato (variable o instrucción) fue accedido recientemente, es muy probable que vuelva a ser accedido en un futuro cercano. Los cachés se benefician de esto al mantener datos reciententes en niveles rápidos de memoria.

Ejemplo en C

En el siguiente bucle, la variable `sum` es accedida y modificada en cada iteración.

```c
#include <stdio.h>
int main()
{
    int arr[1000];
    long long sum = 0;
    for (int i = 0; i < 1000; i++)
    {
        arr[i] = i;
        sum += arr[i];
    }

    printf("Suma = %lld\n", sum);
    return 0;
}
```

b) Localidad espacial.
La **localidad espacial** indica que si se accede a una dirección de memoria, es probable que pronto se acceda a direcciones cercanas. Los cachés aprovechan esto cargando bloques completos de memoria, no solo el byte solicitado.

Ejemplo en C

El recorrido secuencial de un arreglo es el ejemplo más representativo: cada elemento ocupa posiciones de memoria contiguas, por lo que al cargar un bloque de caché se traen varios elementos de una vez.

```c
#include <stdio.h>
#define N 1024
int main()
{
    int matrix[N][N];
    long long sum = 0;
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            sum += matrix[i][j];
        }
    }

    printf("Suma = %lld\n", sum);
    return 0;
}
```

4. Se tiene un sistema con direcciones de memoria de 32 bits y un caché de 8KB con bloques de 32 bytes.

   a) Para un caché de mapeo directo:

   - ¿Cuántos bloques tiene el caché?
     $\text{Bloques} = \frac{\text{tamaño caché}}{\text{tamaño bloque}} = \frac{8\text{KB}}{32\text{B}} = \frac{8192}{32} = 256 \text{ bloques}$
   - ¿Cuántos bits se usan para el offset?
     $\text{Bits offset} = log_2(\text{tamaño bloque}) = log_2(32) = 5 \text{ bits}$
   - ¿Cuántos bits para el índice?
     $\text{Bits índice} = log_2(\text{número de bloques}) = log_2(256) = 8 \text{ bits}$
   - ¿Cuántos bits para la etiqueta?
     $\text{Bits etiqueta} = 32 - \text{bits índice} - \text{bits offset} = 32 - 8 - 5 = 19 \text{ bits}$

   b) Para un caché 4-way set associative:

   - ¿Cuántos conjuntos hay?
     $\text{Conjuntos} = \frac{\text{número de bloques}}{\text{vías}} = \frac{256}{4} = 64 \text{ conjuntos}$
   - ¿Cuántos bits se usan para el índice de conjunto?
     $\text{Bits índice} = log_2(64) = 6 \text{ bits}$
   - ¿Cuántos bits para la etiqueta?
     $\text{Bits etiqueta} = 32 - 6 - 5 = 21 \text{ bits}$

   c) Explicar cómo cambia la probabilidad de conflicto entre mapeo directo y 4-way.
   En el **mapeo directo**, cada bloque de memoria solo puede ubicarse en un único bloque del caché, Si dos bloques frecuentemente usados mapean al mismo índice, se producen conflictos de caché: cada vez que se accede a uno, el otro es desalojado, generando miss tras miss (trashing). Esto ocurre aunque el 99% del caché esté vacío.

   En el **4-way**, cada bloque de memoria puede ubicarse en cualquiera de 4 vías dentro de su conjunto. Así, hasta 4 bloques que mapeen al mismo conjunto pueden coexistir simultáneamente. La probabilidad de conflicto se reduce significativamente porque la política de reemplazo puede elegir qué bloque desalojar, evitando el trashing.

5. Se tiene un caché de 16KB, bloques de 64 bytes, direcciones de 32 bits, y organización 2-way set associative. Calcular el overhead total considerando que cada bloque tiene un bit para indicar si es válido y dos bits de protección.

Parámetros del caché

Número de bloques $= \frac{16\text{KB}}{64\text{B}} = \frac{16384}{64} = 256$ bloques

Número de conjuntos $= \frac{256 \text{ bloques}}{2 \text{ vías}} = 128$ conjuntos

Bits de dirección

Bits offset $= log_2(64) = 6$ bits

Bits índice $= log_2(128) = 7$ bits

Bits etiqueta $= 32 - 7 - 6 = 19$bits

Bits de overhead por bloque

Cada bloque almacena:

- 1 bit válido
- 2 bits de protección
- 19 bits de etiqueta

Overhead por bloque $= 1 + 2 + 19 = 22$ bits

Overhead total

Overhead total = overhead por bloque $\times$ número total de bloques

Overhead total $= 22$ bits $\times$ 256 bloques $= 5,632$ bits $= 704$ bytes